DOCUMENTACIÓN TÉCNICA

Servicio de Inventario

Microservicio #6 – CRUD de Productos

Proyecto de Arquitectura Orientada a Servicios (SOA)

Alumno: Jesus Alexander Hernandez Hernandez

Maestra: María de los Ángeles López Atondo

Sistema: Tienda en línea

Módulo: Inventario

Servicio: Registrar, consultar, actualizar y borrar productos

Versión: 1.0

Fecha: Septiembre de 2026

Contenido

1. Introducción 4
2. Objetivos 4
2.1 Objetivo general 4
2.2 Objetivos específicos 4
3. Alcance y responsabilidades 5
4. Tecnologías utilizadas 5
5. Arquitectura del servicio 6
6. Estructura del proyecto 7
7. Modelo de datos 7
7.1 Tabla productos 7
7.2 Tabla proveedores 8
7.3 Tabla stock 8
8. Creación de la base de datos 8
9. Variables de entorno y seguridad 8
10. Instalación local 9
11. Endpoints de la API 9
12. Contrato de producto 10
12.1 Ejemplo de entrada 10
12.2 Ejemplo de respuesta 10
13. Validaciones implementadas 10
14. Manejo de errores y códigos HTTP 11
15. Swagger / OpenAPI 11
16. Health Check 11
17. Integración con los demás servicios 12
18. Escenarios de integración 12
18.1 Escenario A: cada integrante ejecuta Inventario localmente 12
18.2 Escenario B: Inventario se ejecuta como servicio centralizado 12
19. Pruebas realizadas 13
20. Configuración CORS 13
21. Control de versiones 13
22. Consideraciones de diseño 14
23. Mejoras futuras 14
24. Conclusiones 14

25. Anexo A – Comandos principales 15
26. Anexo B – Resumen para consumidores 15

1. Introducción

El presente documento describe el diseño, implementación, configuración, ejecución, pruebas e integración del Servicio de Inventario desarrollado para una tienda en línea bajo un enfoque de Arquitectura Orientada a Servicios (SOA).

Dentro de la división funcional del proyecto, este componente corresponde al servicio #6 del módulo de Inventario y tiene como responsabilidad administrar el ciclo de vida de los productos mediante operaciones CRUD: registrar, consultar, actualizar y borrar.

El servicio fue diseñado como una API REST independiente. Otros componentes del sistema pueden consumir sus funciones mediante solicitudes HTTP y respuestas JSON, evitando el acceso directo de otros servicios a las credenciales o a la lógica interna de la base de datos.

2. Objetivos

2.1 Objetivo general

Implementar un servicio de inventario desacoplado que permita administrar productos de la tienda en línea y que pueda integrarse con los demás servicios del proyecto mediante una interfaz REST documentada.

2.2 Objetivos específicos

- Registrar nuevos productos en la base de datos.
- Consultar la lista completa de productos.
- Consultar un producto específico mediante su identificador.
- Actualizar la información de un producto existente.
- Eliminar productos existentes.
- Validar los datos recibidos antes de ejecutar operaciones sobre MySQL.
- Proporcionar respuestas JSON y códigos HTTP consistentes.
- Documentar el contrato de la API mediante Swagger/OpenAPI.
- Permitir que otros servicios consuman Inventario sin conocer sus credenciales de base de datos.

3. Alcance y responsabilidades

El Servicio de Inventario implementa exclusivamente las funciones asignadas al CRUD de productos. La separación de responsabilidades es importante para evitar duplicar la lógica perteneciente a otros integrantes del proyecto.

Función	**Responsabilidad**
CRUD de productos	Servicio #6 – incluido en este proyecto
Reporte de existencias	Servicio #7 – fuera del alcance
Alerta de stock mínimo	Servicio #8 – fuera del alcance
Stock máximo, descuentos y ofertas	Servicio #9 – fuera del alcance
Comentarios y reseñas	Servicio #10 – fuera del alcance
CRUD de proveedores	Servicio #11 – fuera del alcance; Inventario solo utiliza la referencia del proveedor

Aunque el esquema local contiene tablas de apoyo como proveedores y stock, esto no significa que este servicio asuma las funciones completas de los servicios responsables de dichas áreas. Se conservan por compatibilidad con el modelo de datos del proyecto.

4. Tecnologías utilizadas

Tecnología	**Uso**
Node.js	Entorno de ejecución del servicio
Express.js	Servidor HTTP y definición de rutas REST
MySQL 8	Persistencia de datos
mysql2	Conexión asíncrona entre Node.js y MySQL
dotenv	Carga de variables de entorno
cors	Configuración de acceso HTTP entre aplicaciones
Swagger / OpenAPI	Documentación interactiva del contrato REST
Git	Control de versiones
GitHub	Repositorio remoto y colaboración
JSON	Formato principal de intercambio de datos

5. Arquitectura del servicio

El proyecto utiliza una estructura por capas para separar las responsabilidades de recepción HTTP, validación, lógica de aplicación y acceso a datos.



Routes define los endpoints públicos. Los middlewares validan parámetros y cuerpos de solicitud. Los controllers gestionan la respuesta HTTP y coordinan la operación. Los services contienen las consultas a la base de datos. MySQL conserva la información persistente.

6. Estructura del proyecto

```
inventario-service/
├── database/
│   └── schema.sql
├── src/
│   ├── config/
│   │   ├── database.js
│   │   └── swagger.js
│   ├── controllers/
│   │   └── producto.controller.js
│   ├── middlewares/
│   │   ├── validarProducto.js
│   │   └── errorHandler.js
│   ├── routes/
│   │   └── producto.routes.js
│   ├── services/
│   │   └── producto.service.js
│   └── app.js
├── .env
├── .env.example
├── .gitignore
├── README.md
├── package.json
└── package-lock.json
```

El archivo .env existe únicamente en cada entorno local y está excluido del repositorio mediante .gitignore.

7. Modelo de datos

7.1 Tabla productos

Campo	Tipo	Descripción
id	INT, PK, AUTO_INCREMENT	Identificador único
nombre	VARCHAR(150)	Nombre del producto; obligatorio
lote	VARCHAR(100)	Lote; opcional
serie	VARCHAR(100)	Número o código de serie; opcional
caducidad	DATE	Fecha de caducidad; opcional
id_proveedor	INT, FK	Referencia a proveedores.id; opcional
costo	DECIMAL(10,2)	Costo del producto; obligatorio
precio	DECIMAL(10,2)	Precio de venta; obligatorio

7.2 Tabla proveedores

La tabla proveedores se incluye como dependencia de integridad referencial para id_proveedor. El CRUD completo de proveedores pertenece al servicio #11.

7.3 Tabla stock

Campo	Tipo	Descripción
id_producto	INT, PK/FK	Producto relacionado
stock_actual	INT	Existencia actual
stock_minimo	INT	Umbral mínimo
stock_maximo	INT	Umbral máximo

La existencia de esta tabla no amplía el alcance del servicio #6 a reportes o alertas de inventario.

8. Creación de la base de datos

El repositorio incluye database/schema.sql para preparar una instalación local compatible. La creación del esquema debe realizarse con una cuenta administrativa de MySQL. La cuenta utilizada por la aplicación no necesita privilegios para crear tablas.

```
CREATE DATABASE IF NOT EXISTS tienda_online
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

USE tienda_online;
```

Después de crear las tablas, la aplicación puede utilizar una cuenta dedicada con permisos limitados a SELECT, INSERT, UPDATE y DELETE. Esto reduce el uso innecesario de una cuenta administrativa durante la ejecución normal.

9. Variables de entorno y seguridad

Las credenciales se almacenan en .env y no se versionan. El repositorio incluye .env.example para indicar las variables necesarias sin publicar secretos.

```
PORT=3000

DB_HOST=localhost
DB_PORT=3306
DB_USER=inventario_app
DB_PASSWORD=TU_CONTRASEÑA
DB_NAME=tienda_online
```

- No subir .env a GitHub.
- No compartir contraseñas de MySQL con servicios consumidores.
- Utilizar una cuenta de aplicación con permisos mínimos necesarios.
- Mantener node_modules fuera del repositorio.
- Usar .env.example únicamente como plantilla.

10. Instalación local

Requisitos: Node.js, npm, MySQL 8 o compatible y Git.

1. Clonar el repositorio:

```
git clone git@github.com:AlexanderHHZ/inventario-service.git
```

2. Entrar al directorio:

```
cd inventario-service
```

3. Instalar dependencias:

```
npm install
```

4. Preparar MySQL utilizando database/schema.sql con una cuenta administrativa.

5. Crear .env a partir de .env.example y completar las credenciales locales.

6. Ejecutar el servicio:

```
npm run dev

# o ejecución normal
npm start
```

Por defecto, el servicio escucha en http://localhost:3000.

11. Endpoints de la API

Método	Endpoint	Descripción	Éxito
GET	/api/health	Comprobar que el servicio HTTP está activo	200
GET	/api/productos	Consultar todos los productos	200
GET	/api/productos/{id}	Consultar un producto por ID	200
POST	/api/productos	Registrar un producto	201
PUT	/api/productos/{id}	Actualizar un producto	200
DELETE	/api/productos/{id}	Eliminar un producto	200

12. Contrato de producto

12.1 Ejemplo de entrada

```
{
  "nombre": "Teclado mecanico RGB",
  "lote": "LOT-001",
  "serie": "TEC-001",
  "caducidad": null,
  "id_proveedor": 1,
  "costo": 700,
  "precio": 999
}
```

Los campos nombre, costo y precio son obligatorios. lote, serie, caducidad e id_proveedor son opcionales. Si se proporciona id_proveedor, debe ser un entero positivo y corresponder a un proveedor existente.

12.2 Ejemplo de respuesta

```
{
  "success": true,
  "message": "Producto encontrado",
  "data": {
    "id": 1,
    "nombre": "Teclado mecanico RGB",
    "lote": "LOT-001",
    "serie": "TEC-001",
    "caducidad": null,
    "id_proveedor": 1,
    "costo": "700.00",
    "precio": "999.00"
  }
}
```

Nota de contrato: MySQL utiliza DECIMAL para costo y precio y el controlador mysql2 puede devolver estos valores como cadenas decimales. Los consumidores deben respetar el contrato documentado y no depender de conversiones implícitas. Antes de la integración definitiva del equipo conviene acordar si estos importes se mantendrán como cadenas decimales o se normalizarán explícitamente.

13. Validaciones implementadas

- El ID de ruta debe ser un número entero positivo.
- nombre debe ser una cadena no vacía.
- costo debe ser numérico y mayor o igual a cero.
- precio debe ser numérico y mayor o igual a cero.
- id_proveedor, cuando se envía, debe ser un entero positivo.
- El proveedor indicado debe existir en la tabla proveedores.
- lote y serie, cuando se envían, deben ser cadenas.
- caducidad, cuando se envía, debe representar una fecha válida.
- El JSON mal formado genera una respuesta controlada con código 400.

14. Manejo de errores y códigos HTTP

Código	Uso en el servicio
200 OK	Consulta, actualización o eliminación correcta
201 Created	Producto registrado correctamente
400 Bad Request	ID, cuerpo, datos o JSON inválidos
404 Not Found	Producto inexistente o ruta no encontrada
500 Internal Server Error	Error interno no controlado

Las respuestas siguen una estructura JSON con la propiedad success y un mensaje descriptivo. Cuando corresponde, también se incluye data.

15. Swagger / OpenAPI

El servicio incorpora documentación interactiva mediante Swagger. Con el servidor en ejecución puede consultarse en:

```
http://localhost:3000/api/docs
```

Swagger describe los endpoints principales, parámetros, cuerpos de solicitud, esquemas y respuestas esperadas. Además permite ejecutar pruebas HTTP desde el navegador, lo cual facilita la integración entre integrantes del equipo.

16. Health Check

El endpoint GET /api/health permite comprobar que el proceso HTTP está funcionando.

```
{
  "success": true,
  "service": "inventario-service",
  "message": "Servicio de inventario funcionando correctamente"
}
```

Este endpoint comprueba la disponibilidad del servicio web. No debe interpretarse por sí solo como una comprobación exhaustiva del estado de MySQL.

17. Integración con los demás servicios

La integración se basa en HTTP y JSON. Un servicio consumidor debe conocer la dirección base del Servicio de Inventario y utilizar los endpoints públicos; no debe ejecutar consultas directas sobre la base de datos de Inventario.

```
Otro servicio
     |
     | GET /api/productos/1
     v
Servicio de Inventario
     |
     | consulta interna
     v
   MySQL
     |
     v
Respuesta JSON al consumidor
```

Este enfoque reduce el acoplamiento. La implementación interna de Inventario puede evolucionar mientras el contrato REST se mantenga estable.

18. Escenarios de integración

18.1 Escenario A: cada integrante ejecuta Inventario localmente

Un compañero puede clonar el repositorio, instalar dependencias, preparar una base de datos compatible, crear su propio .env y ejecutar el servicio. En este escenario cada equipo de desarrollo dispone de una instancia local.

- Clonar el repositorio.
- Ejecutar npm install.
- Crear/preparar MySQL con el esquema compatible.
- Crear .env con credenciales locales.
- Ejecutar npm start o npm run dev.
- Consumir la API en el puerto configurado.

El nombre de la base de datos puede adaptarse en DB_NAME siempre que las tablas utilizadas sean compatibles. El script proporcionado crea tienda_online como configuración de referencia.

18.2 Escenario B: Inventario se ejecuta como servicio centralizado

Inventario puede desplegarse o ejecutarse en un equipo/servidor accesible por la red. En este caso los demás servicios no necesitan instalar su base de datos ni conocer sus credenciales.

```
INVENTARIO_BASE_URL=http://servidor:3000

GET {INVENTARIO_BASE_URL}/api/productos
GET {INVENTARIO_BASE_URL}/api/productos/1
```

Los consumidores solo requieren la URL base y el contrato de la API. Las credenciales de MySQL permanecen privadas en el entorno donde se ejecuta Inventario.

19. Pruebas realizadas

Se ejecutaron pruebas manuales de extremo a extremo contra la API y MySQL.

Prueba	Resultado esperado	Estado
GET /api/productos	200 y lista desde MySQL	Correcto
POST /api/productos	201 y producto creado	Correcto
GET /api/productos/{id}	200 y producto solicitado	Correcto
PUT /api/productos/{id}	200 y datos actualizados	Correcto
DELETE /api/productos/{id}	200 y eliminación confirmada	Correcto
GET producto eliminado/inexistente	404 Producto no encontrado	Correcto
DELETE inexistente	404	Correcto
ID abc o negativo	400	Correcto
JSON mal formado	400 JSON inválido o mal formado	Correcto
CORS	Encabezado de acceso presente	Correcto
Swagger	Documentación accesible	Correcto
Usuario MySQL dedicado	CRUD funcional sin usar root	Correcto

20. Configuración CORS

Durante el desarrollo se utiliza cors() con configuración abierta, lo que facilita las pruebas y la integración local entre servicios. Para un despliegue real se recomienda restringir los orígenes permitidos de acuerdo con la infraestructura final.

21. Control de versiones

El proyecto utiliza Git y GitHub. .gitignore excluye dependencias, variables de entorno, logs y archivos de sistema.

```
node_modules/
.env
*.log
npm-debug.log*
.DS_Store
Thumbs.db
```

El repositorio remoto contiene el código fuente, el esquema, la plantilla de configuración y la documentación, pero no las credenciales locales.

22. Consideraciones de diseño

El modelo oficial del proyecto ubica lote, serie y caducidad en PRODUCTOS. Se respeta dicho diseño para mantener compatibilidad con el trabajo del equipo. En un sistema de producción podría evaluarse separar la definición del producto de lotes o unidades físicas, especialmente cuando existen múltiples lotes, números de serie o fechas de caducidad para un mismo producto.

También debe definirse en conjunto el tratamiento de importes DECIMAL en el contrato JSON. Para sistemas monetarios es importante evitar pérdidas de precisión; por ello cualquier cambio de representación debe acordarse entre productores y consumidores de la API.

23. Mejoras futuras

- Agregar pruebas automatizadas de integración y de endpoints.
- Agregar paginación y filtros a la consulta de productos si el volumen crece.
- Incorporar autenticación/autorización cuando el servicio de control de acceso esté integrado.
- Restringir CORS para ambientes de despliegue.
- Agregar comprobación de disponibilidad de MySQL a un endpoint de readiness separado.
- Definir y versionar formalmente el contrato compartido entre servicios.
- Realizar una prueba real de integración con al menos otro microservicio del proyecto.

24. Conclusiones

El Servicio de Inventario cumple la responsabilidad asignada al módulo #6 mediante una API REST que implementa las operaciones CRUD de productos. La solución utiliza Node.js, Express y MySQL, incluye validaciones, manejo de errores, documentación Swagger, variables de entorno y control de versiones.

La principal característica de integración es que los demás componentes interactúan con Inventario a través de HTTP/JSON en lugar de acceder directamente a su base de datos. Esto conserva la separación de responsabilidades propia de una arquitectura orientada a servicios y permite que cada integrante desarrolle su módulo con un contrato de comunicación común.

El servicio ha sido probado para operaciones exitosas y escenarios de error relevantes. La siguiente etapa del proyecto consiste en validar la integración con los demás servicios y acordar los detalles finales del contrato compartido antes de la entrega.

25. Anexo A – Comandos principales

```
# Desarrollo
npm run dev

# Ejecución normal
npm start

# Estado de Git
git status

# Agregar cambios
git add .

# Crear commit
git commit -m "Descripción del cambio"

# Subir cambios
git push
```

26. Anexo B – Resumen para consumidores

Dato	Valor
Protocolo	HTTP
Formato	JSON
Ruta base de productos	/api/productos
Documentación	/api/docs
Health check	/api/health
Credenciales MySQL para consumidores	No requeridas
Acceso directo a BD desde otros servicios	No recomendado; consumir la API